



Today's agenda

↳ Man not Pair

↳ Man not Subarray

↳ Amazon Berlin quest



AlgoPrep



Q) max xor Pair

↳ Given arr[N], find max xor Pair.

Ex: arr[4] : 4 3 2 7

$$4^3 = 7 \rightarrow \text{ans} = 7$$

$$4^7 = 3$$

Idea 1

↳ iterate and get every Pair, Pick the max one.

T.C: $O(N^2)$

S.C: $O(1)$

Idea 2

arr[9]: 22 61 38 27 21 34 42 37 43

22: 0 1 0 1 1 0 X

61: 1 1 1 1 0 1 XX

38: 1 0 0 1 1 0 XXX

27: 0 1 1 0 1 1 X

21: 0 1 0 1 0 1 X

34: 1 0 0 0 1 0 XXX

42: 1 0 1 0 1 0 XXXX

37: 1 0 0 1 0 1 XXX

43: 1 0 1 0 1 1 ✓

$a^b = \text{max}$

$a = 22$, find b in the array

22: 0 1 0 1 1 0

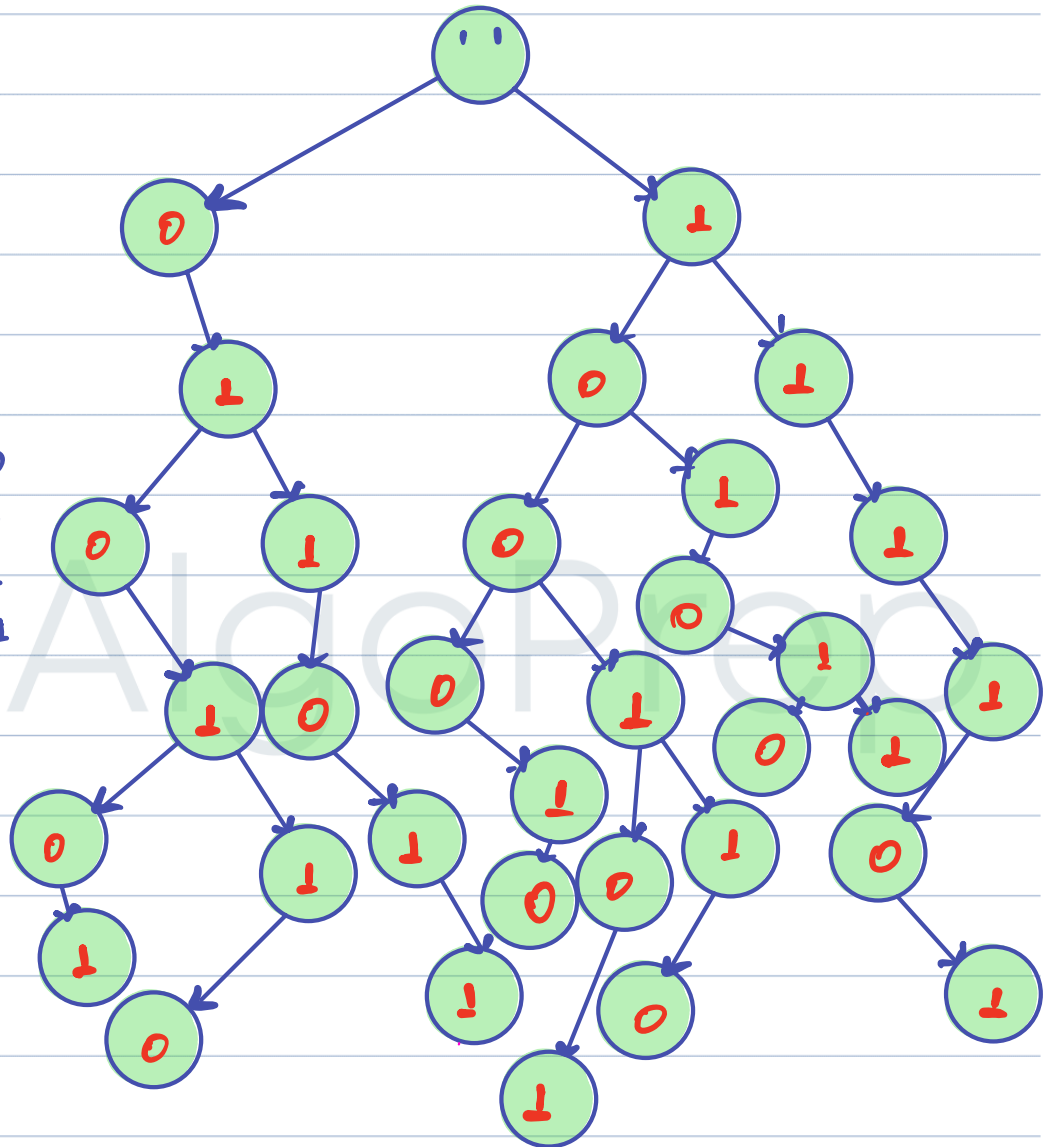
b: 1 0 1 0 1 1

$$O(32 \cdot N) \approx N \approx O(32N^2)$$



↳ To find the best 'b' val for given 'a', build tree for all the array elements.

22: ⁵0 ⁴1 ³0 ²1 ¹1 0
61: 1 1 1 1 0 1
38: 1 0 0 1 1 0
27: 0 1 1 0 1 1
21: 0 1 0 1 0 1
34: 1 0 0 0 1 0
42: 1 0 1 0 1 0
→ 37: 1 0 0 1 0 1
43: 1 0 1 0 1 1



22: ⁵0 ⁴1 ³0 ²1 ¹1 0

b: 1 0 1 0 1 1
1 1 1 1 0 1

61: 1 1 1 1 0 1

b: 0 1 0 1 1 0
1 0 1 0 1 1

38: 1 0 0 1 1 0

b: 0 1 1 0 1 1
1 1 1 1 0 1



// Pseudo code

```
class Node{  
    HashMap < Integer, Node > hm;  
}
```

```
Node root = new Node();
```

```
main ( ) {  
    int l[] arr;  $\rightarrow$  input  
    o(1)  $\leftarrow$  int n = arr.length;  
    o(32)  $\leftarrow$  int P = getMSB(n);
```

```
    for (int i=0; i<arr.length; i++) {  
        o(N+32)  $\leftarrow$  insert(arr[i], P);  
    }
```

TC: $O(32 \times N)$

SC: $O(32 \times N)$

```
        int ans = 0;  
        for (int i=0; i<arr.length; i++) {  
            int a = arr[i];  
            int b = search(arr[i], P);  
            int xor = a ^ b;  
            ans = math.max(ans, xor);  
        }  
        s.o.p(ans);  
    }
```



```
void insert (int a, int p){
```

```
    Node t = root;
```

```
    for (int i = p; i >= 0; i--){
```

```
        int ch = checkBit(a, i);
```

```
        if (t.hm.containsKey(ch) == true){
```

```
            t = t.hm.get(ch);
```

```
        }
```

```
        else{
```

```
            Node n1 = new Node();
```

```
            t.hm.put(ch, n1);
```

```
            t = t.hm.get(ch);
```

```
        }
```

```
    }
```

```
}
```



```
int search ( int a, int p ) {
```

```
    Node t = root;
```

```
    int b = 0;
```

```
    for (int i = p; i >= 0; i--) {
```

```
        int mybit = checkBit(a, i);
```

```
        int ch = 1 - mybit;
```

```
        if ( t.hm.containsKey(ch) == true ) {
```

```
            b = b + (2i * ch);
```

```
            t = t.hm.get(ch);
```

```
        } else {
```

```
            b = b + (2i * mybit);
```

```
            t = t.hm.get(mybit);
```

```
        }
```

```
    return b;
```

```
}
```



Q) max xor subarray

↳ Find the subarray with max xor value.

arr[4] : 4 3 2 7

{4, 3} {4, 2, 7}

Idea 1

↳ Create all the subarrays and pick with max xor value.

T.C: $O(N^2)$

S.C: $O(1)$

Idea 2 → $prefix[i] = xor(0 \dots i);$

arr[5] : ⁰ x1 ¹ x2 ² x3 ³ x4 ⁴ x5

prefix[5] : a b c d e

↳ $xor(1, 3) = prefix[3] \oplus prefix[0]$

~~(x1 ^ x2 ^ x3 ^ x4)~~ ^ ~~x1~~

↳ $xor(2, 5) = prefix[5] \oplus prefix[1]$

~~(x1 ^ x2 ^ x3 ^ x4 ^ x5)~~ ^ ~~(x1 ^ x2)~~



arr[5] : ⁰ n1 ¹ n2 ² n3 ³ n4 ⁴ n5

pnor[5] : a b c d e

$$\rightarrow \text{pnor}[1] \wedge \text{pnor}[3] = \text{xor}(2, 3)$$

$$\rightarrow \text{pnor}[1] \wedge \text{pnor}[4] = \text{xor}(2, 4)$$

↳ xor of any 2 nos in pnor[] gives you xor of subarray of original array.

↳ man nos subarray of original array =

man nos pair in pnor array.



↳ list of urls were given.

alena / 1234 / e

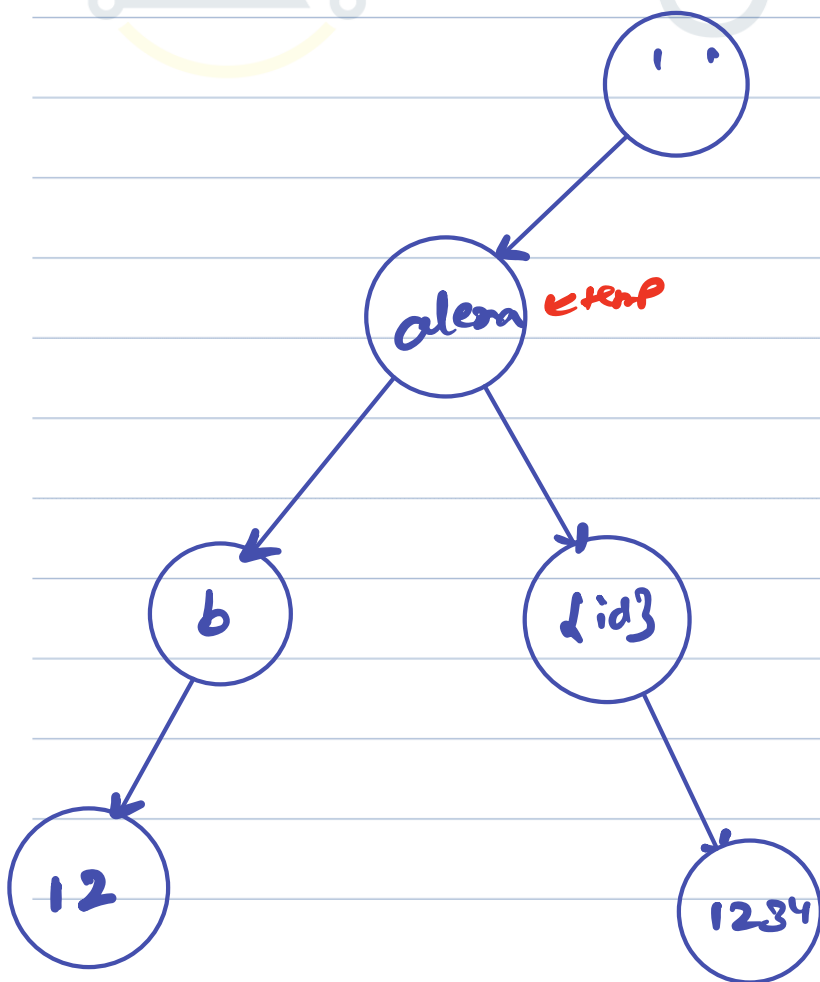
alena / {id} / 1234

apple / h / {id} / 12



→ query urls

↳ alena / b / 123



alena / b / 1234



→ Autocomplete.



AlgoPrep